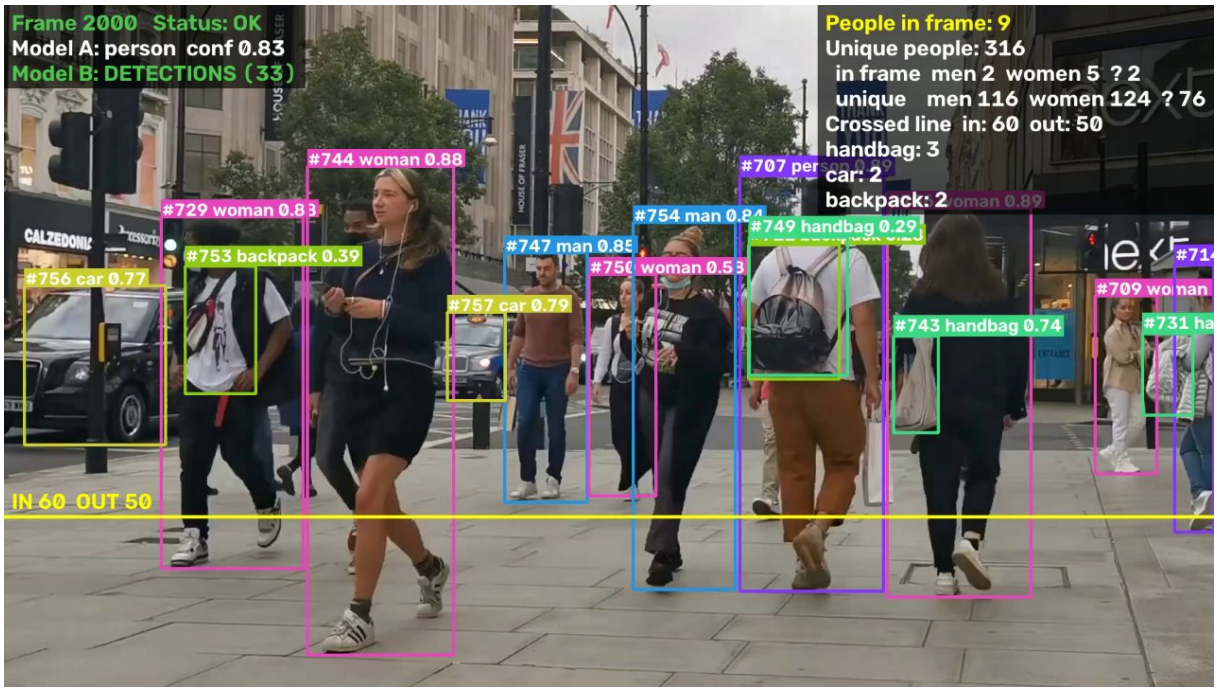
vpipe: CPU Video Analytics

A learning guide: how the project works, step by step

This guide explains the **Real-Time Video Inference Pipeline** (`vpipe`) for someone who wants to **understand** it, not just run it. It starts with the big picture, then follows a single video frame through every part of the system, explains the key computer-vision and parallel-programming ideas, and ends with commands, outputs, measured results, a code map, exercises and a glossary.



A frame of the output video: every person has a tracking ID and a man/woman estimate, bags and cars are detected, the yellow line counts people crossing, and the panels show live counts.

Chapter	What you will learn
1. The big picture	What goes in, what comes out, and the two AI models
2. Follow one frame	The 7 steps a frame goes through, with the source file for each
3. Key ideas	Detection vs classification, tracking, processes vs threads, queues, ordering
4. Modes	Accurate, fast (30 FPS) and assignment mode, with measured trade-offs
5. Setup and run	Install, run, and use VS Code
6. Outputs	Every output file and how to read the video
7. Performance lessons	Real bottlenecks found while making it fast
8. Code map + exercises + glossary	Where to start reading, things to try, terms

1. The big picture

You give vpipe a street video. It gives you back the same video with boxes drawn on people, bags and vehicles, plus numbers: how many people are in each frame, how many *different* people appeared, how many crossed a line, and an estimate per person of man / woman / unknown and whether they carry a bag.

Input	Output
A video file, e.g. `data/people_walking.mp4` (1920x1080, 30 frames per second, 117 s = 3,493 frames)	An annotated MP4, CSV/JSON files with counts per frame and per person, charts and heatmaps

Three AI models do the seeing

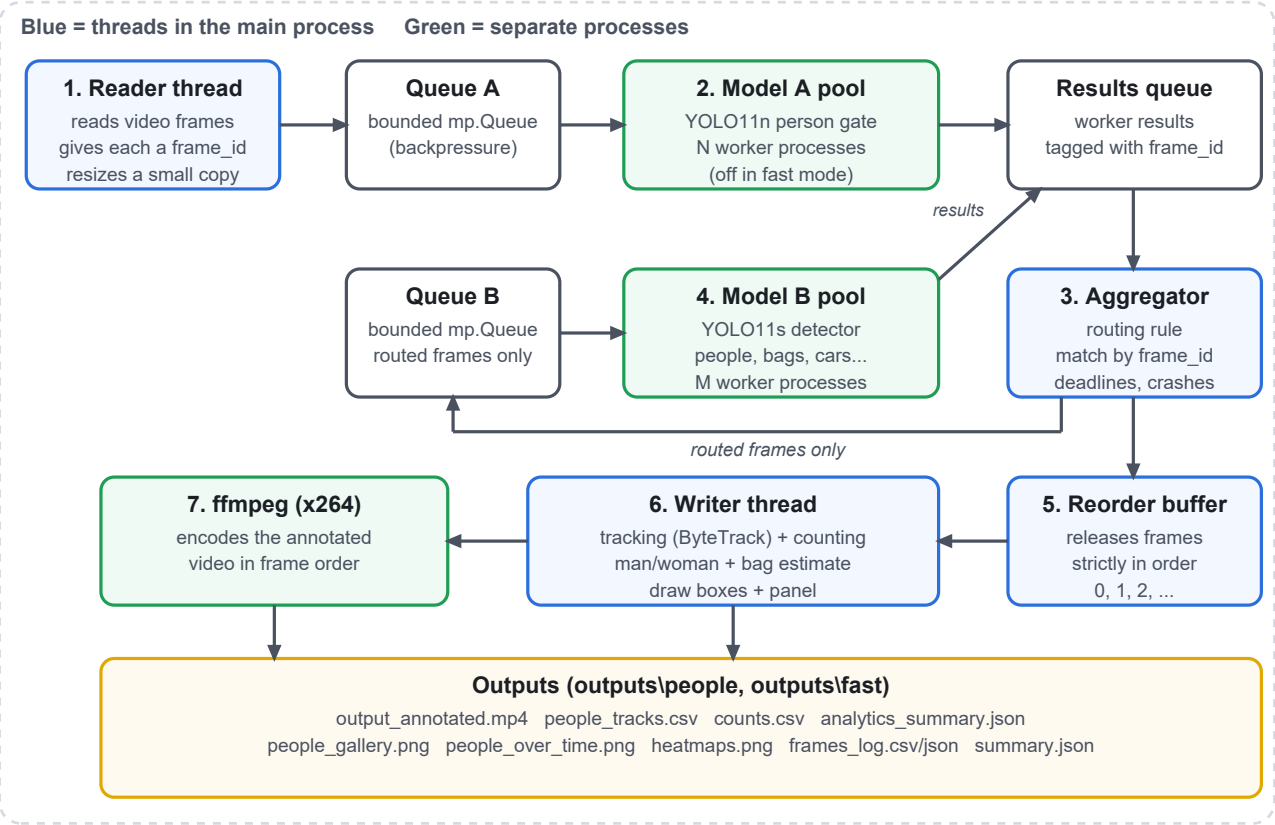
Model	What it answers	Where it runs
Model A - YOLO11n (small, 320 px)	"Is there a person in this frame?" (the *gate*)	Pool of worker processes
Model B - YOLO11s (accurate, 640 px)	"Where is every person, bag, car...?" (boxes + labels, 80 object classes)	Pool of worker processes
Attributes - OpenVINO person-attributes-recognition-crossroad-0230	"Does this person look male? carry a bag? a backpack? a hat?"	Background threads in the main process

Around the models sits the **pipeline**: the code that reads frames, moves them between processes, keeps them in order, tracks people over time, counts, draws and writes the video. Most of what you can learn from this project is in the pipeline, not in the models.

> **Why not just run YOLO in a loop?**
> A simple loop (read frame, run models, draw, write) uses one CPU core and waits for each step. vpipe runs the models in several processes at once, keeps every core busy, survives a crashing worker, bounds its memory, and still writes frames in perfect order. That is what makes it a *pipeline*.

2. Follow one frame through the pipeline

The diagram shows the whole system. Blue boxes are threads inside the main Python process; green boxes are separate processes. Follow the numbers.



The vpipe pipeline. Every message carries the frame's `frame_id`, so results can always be matched to the right frame.

Step	Source file(s)	What happens
1. Reader thread	`reader.py`	Reads the next frame from the video, gives it a number (`frame_id`) and a timestamp, keeps the full-size original in memory (FrameStore), and puts a **small resized copy** into Queue A. Small copies are cheaper to send between processes. If memory is full, it **waits** (backpressure).
2. Model A pool (the gate)	`workers.py`, `pools.py`, `models.py`	Each worker process loads its own YOLO11n, takes a task from Queue A, answers "person or not", and puts the result (with the same frame_id) into the results queue. Workers never decide anything else.
3. Aggregator (the brain)	`aggregator.py`, `routing.py`	Runs in the main thread and is the **only owner** of each frame's state. It applies the routing rule (person found with confidence >= 0.30 -> send to Model B), matches every result by frame_id, enforces timeouts, and notices crashed workers.
4. Model B pool (the detector)	`workers.py`, `models.py`	Gets a 640 px copy of routed frames and returns boxes: label, confidence and position for every object it finds.
5. Reorder buffer	`aggregator.py`	Workers finish in any order (frame 12 may finish before frame 10). The buffer releases frame *k* only after frames 0...k-1 are done, so the output video is always in order.
6. Writer thread	`writer.py`, `analytics.py`, `attributes.py`	Tracks people across frames (ByteTrack gives each person an ID), counts, checks line crossings, estimates man/woman and bags, and draws boxes and panels. Drawing runs on a small thread pool.

Step	Source file(s)	What happens
7. ffmpeg encoder	`writer.py`	A separate ffmpeg process (x264) receives the drawn frames in order and writes the H.264 MP4. Logs, CSVs, charts and summaries are written at the end.

3. Key ideas explained

Classification vs detection

A **classifier** looks at a whole image and gives one label ("dog", 0.91). A **detector** finds every object and draws a box around each ("person" at x,y,w,h, 0.87). vpipe's original assignment mode used a classifier (YOLO11n-cls, trained on ImageNet), which has **no "person" class at all**, so it could never route a frame. The people mode therefore uses a small *detector* as the gate.

Confidence and thresholds

Every prediction has a confidence between 0 and 1. A **threshold** decides what counts: the gate needs >= 0.30; the detector keeps boxes >= 0.10 so the tracker can still follow people who are partly hidden.

Tracking: from boxes to people

A detector only sees single frames. **ByteTrack** links boxes from frame to frame using position and motion, so the same person keeps the same ID (#12). That makes it possible to count **unique** people (a track that lived >= 15 frames) and **line crossings** (the foot point crossing the yellow line). Tracking must see frames in order, which is why it runs in the writer, after the reorder buffer.

Averaging over a track

The man/woman model is unreliable on a single crop (pose, blur, seen from behind). vpipe samples up to 12 crops per person, averages the probabilities, skips crops that are too small or cut off by the frame edge, and says **unknown** when the average stays near the middle. A good general lesson: *aggregate noisy predictions over time instead of trusting one frame*.

Processes, threads and the GIL

Python's **GIL** lets only one thread run Python code at a time, so threads cannot run models in parallel. vpipe runs models in **separate processes** (true parallelism, and a crashing worker cannot take down the main program). Inside the main process, threads are still useful for work that releases the GIL: video decoding, OpenCV drawing, OpenVINO inference and the ffmpeg pipe.

Bounded queues and backpressure

If the reader were faster than the models, frames would pile up until memory ran out. Every queue is **bounded** and the number of frames in memory is capped (64 slots). When everything is full the reader waits. This is called **backpressure**. In *live* mode the reader never waits; a frame that does not fit is marked DROPPED.

Every frame gets an honest status

Status	Meaning
OK / DETECTIONS / SKIPPED	Normal results (SKIPPED = the gate said no person)

Status	Meaning
HELD	Fast mode: not analysed on purpose; the tracker repeats the last boxes
NOT_GATED	Fast mode: analysed by Model B without asking the gate first
ERROR / TIMEOUT / CRASHED	A model failed for this frame; shown in red, counts become UNAVAILABLE (never 0)
DROPPED / INVALID / ABORTED	Live overload / unreadable frame / program stopped

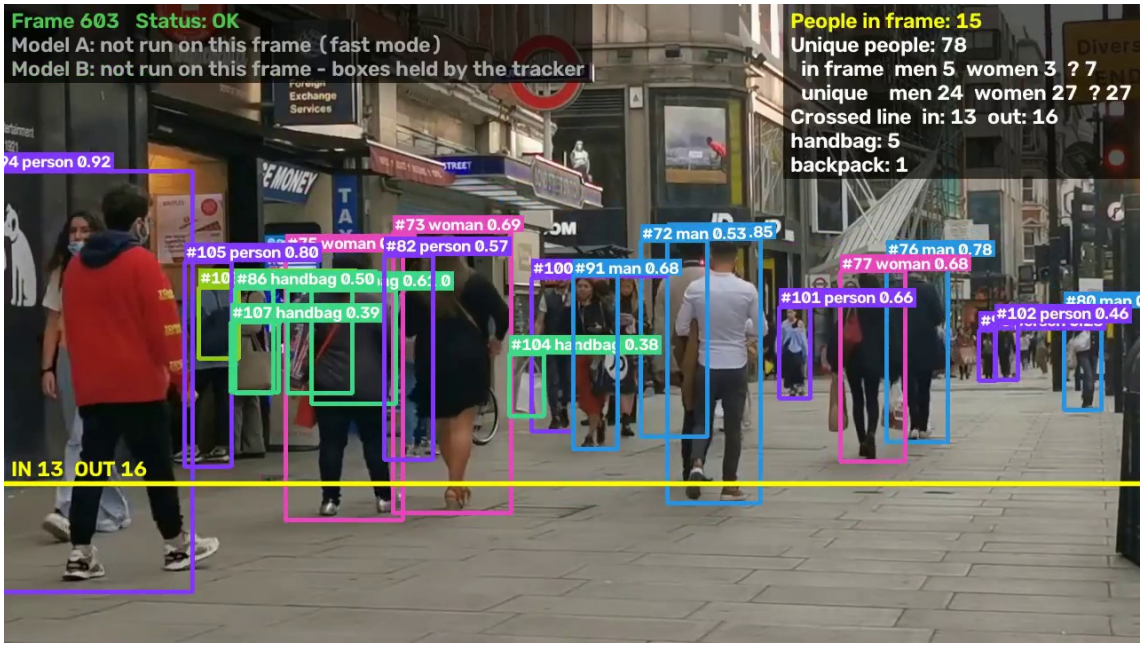
4. The three ways to run it

Mode	Command	What it does
Accurate (people preset)	`.\predict full`	Gate on every frame, detector on every 2nd frame (tracker holds boxes in between), 1080p video
Fast (30 FPS)	`.\predict fast`	First 40 s; models on every 8th frame (4-core CPU), no gate, 720p video, never faster than the video's 30 FPS
Assignment	`python -m vpipe run ...`	Original task: ImageNet classifier as Model A, YOLO11n as Model B

Measured on a 4-core Intel i5-8265U running at 1.6 GHz

	Every frame	Accurate (default)	Fast
Speed	4.3 FPS	6.3 FPS	26.6 FPS
Time	13 min 35 s (full video)	9 min 12 s (full video)	45 s for the 40 s clip
Unique people	495 (full video)	423 (full video)	138 (40 s; accurate on 40 s: 167)
Per-frame people / handbags (40 s clip)	-	15.2 / 2.2	13.3 / 1.2

> **The trade-off to remember**
> Analysing fewer frames is the biggest speed lever, but small, short-lived objects (like handbags) are missed more often. Per-frame people counts barely move; totals that depend on tracking move more. Choose the mode by what matters: speed or counts.



Fast mode: a HELD frame. The top-left panel says Model A/B did not run on this frame; the boxes come from the tracker.

5. Setup and run

One-time install (PowerShell)

```
git clone <your-repo-url> data_pipeline
cd data_pipeline
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install torch torchvision `
    --index-url https://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

Download the attributes model (3 MB) and the demo video

```
$n = "person-attributes-recognition-crossroad-0230"
$u = "https://storage.openvinotoolkit.org/repositories/open_model_zoo/2023.0/" + `
     "models_bin/1/$n/FP32/$n"
New-Item -ItemType Directory -Force models | Out-Null
Invoke-WebRequest "$u.xml" -OutFile "models\$n.xml"
Invoke-WebRequest "$u.bin" -OutFile "models\$n.bin"

.\.venv\Scripts\python.exe -m pip install yt-dlp
.\.venv\Scripts\python.exe -m yt_dlp -f "bv*[vcodec^=avc1][height<=1080][ext=mp4]" `
    -o data/people_walking.mp4 "https://youtu.be/YzcawvDGe4Y"
```

Run

```
.\predict            # first 10 s, accurate mode       -> outputs\test\people
.\predict fast       # first 40 s, fast mode (30 FPS)  -> outputs\fast
.\predict full       # whole video, accurate mode       -> outputs\people
.\.venv\Scripts\python.exe -m pytest -o addopts= -q   # 41 tests with fake models
```

The output video opens automatically when a run finishes. While it runs you see a line every 5 seconds:

```
progress: 1200/3493 frames (34%)  6.4 fps  ETA 6:00
```

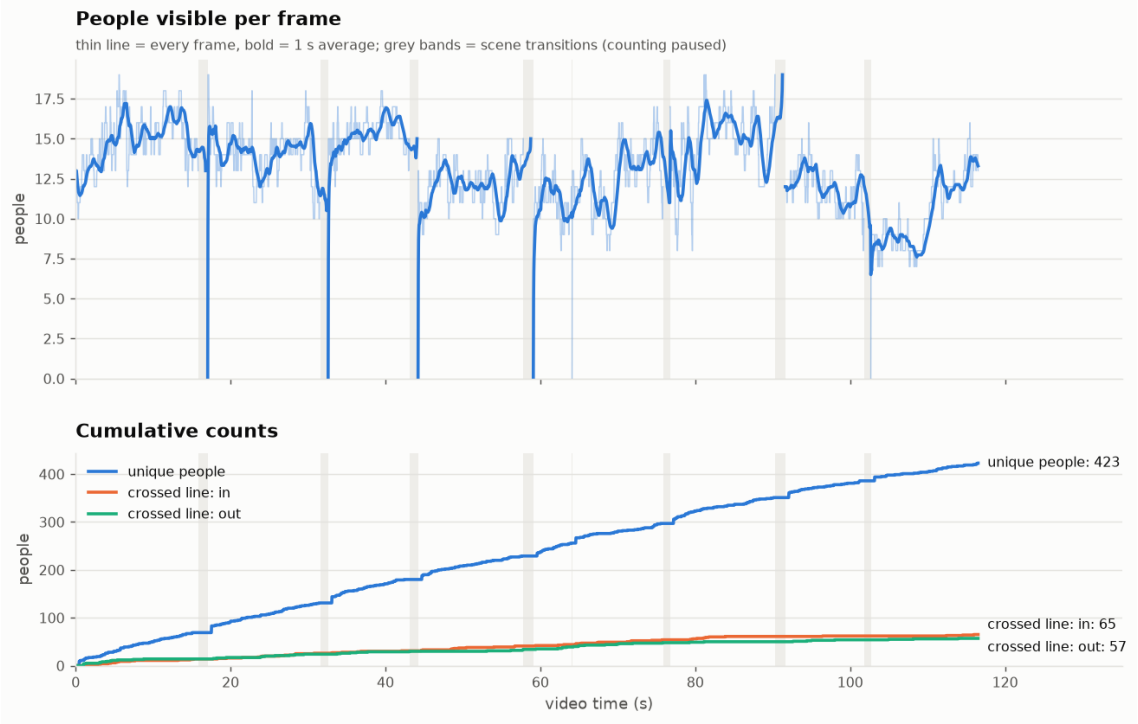
Common mistakes
No module named vpipe - you used the system Python; use `.\.venv\Scripts\python.exe` or `.\predict`.
Typing run in PowerShell - another program owns that name; type `.\predict`.
Two runs at once - they share the CPU and the second one into the same folder is refused.
Video open in a player - the new video gets a date/time in its name; close the player.

In VS Code: Ctrl+Shift+B runs the full video; Terminal > Run Task has the quick test, fast mode, the tests and Open output video; Ctrl+Shift+D then F5 runs with the debugger (slower).

6. Outputs and how to read them

File	What it contains
output_annotated.mp4	The video with boxes, IDs, man/woman labels, the counting line and live panels
people_tracks.csv	One row per unique person: ID, scene, gender estimate, p(male), bag, backpack, hat
counts.csv	One row per frame: people, men/women/unknown, unique so far, line in/out, every object class
analytics_summary.json	All totals: unique people, peak, line crossings, per scene, per object class
people_gallery.png	Sample crops grouped as men / women / unknown - check the labels by eye
people_over_time.png / heatmaps.png	Chart of people over time / where people walked in each scene
frames_log.csv / .json, summary.json	Technical log per frame (statuses, timings) and run summary (FPS, latency, CPU, memory)

On screen	Meaning
#12 man 0.87 (blue box)	Person with tracking ID 12, estimated man; 0.87 = detection confidence
#7 woman 0.91 (pink box)	Person 7, estimated woman
#3 person 0.80	Man/woman not certain (unknown)
handbag / backpack / car ...	Detected objects
Yellow line, IN / OUT	People crossing downward (in) or upward (out)
Red border, ERROR	Something failed for this frame; it is shown, never hidden



people_over_time.png from a full run: people per frame (top) and cumulative unique people and line crossings (bottom). Grey bands are scene changes, where counting pauses.

7. Performance lessons (what was actually slow)

Making the pipeline fast was mostly about **measuring** before changing anything. These are the real findings from this project, and they apply to many other Python programs:

Symptom	Real cause (measured)	Fix
Detector workers idle, yet speed stuck	Model A (one gate worker) was the bottleneck: its queue was always full	Detect on every 2nd frame; in fast mode skip the gate (+18%)
All 8 cores at 100% while workers had nothing to do	PyTorch's OpenMP threads spin-wait after each inference	OMP_WAIT_POLICY=PASSIVE: CPU use fell from 587% to 237%
Writer capped at ~11 FPS	Every frame was resized and drawn one at a time; a 1080p resize takes 19 ms on one 1.6 GHz core	Draw on a 3-thread pool, encode in order; shrink to 720p before drawing
Man/woman model slowed every frame	It ran inline in the ordered writer	Run it on background threads
868 MB video	Windows' built-in H.264 encoder ignores quality settings	ffmpeg/x264 CRF 23: 86 MB
Broken video file	Two runs wrote the same file at the same time	A lock file refuses a second run into the same folder
Everything slower than expected	This laptop's CPU never turbos (stays at 1.6 GHz)	Not fixable in code; enabling Turbo Boost would give up to ~2x

> **Tools used**
> Per-stage timers in the writer and analytics (logged at the end of each run), queue-depth sampling in summary.json, `py-spy` (a sampling profiler that shows which thread and line takes the time), and Windows performance counters for the real CPU clock.

Experiments that did **not** help (also worth learning from): a smaller detector (YOLO11n) was as fast as frame skipping but lost 39% of handbags; OpenVINO export of YOLO gave +15% speed but 6x slower start-up; more gate workers and a smaller gate image gave nothing because the CPU was already full; a CLIP model for man/woman was less accurate and 20x slower.

8. Code map: where to start reading

File (src/vpipe/)	Read it to learn
cli.py	Command-line options, the people preset, fast mode and how settings follow the CPU size
pipeline.py	How everything is wired together: processes, threads, main loop, shutdown, progress
reader.py	Reading frames, frame IDs, HELD/NOT_GATED frames, backpressure, live pacing
aggregator.py	**Start here** for the core logic: routing, matching by frame_id, timeouts, reordering (pure Python, no processes, easy to test)
workers.py / pools.py	Worker processes, thread settings, crash detection and respawn
models.py	YOLO wrappers and fake models used in tests
analytics.py	Scene changes, ByteTrack, unique counting, line crossing, heatmaps, charts
attributes.py	Man/woman and bag estimates, averaging per track, background threads
writer.py	Drawing overlays, parallel rendering, the ffmpeg encoder, logs
schema.py / config.py	All statuses and all settings in one place

Suggested reading order: **schema.py** -> **aggregator.py** + its tests -> **pipeline.py** -> **analytics.py** -> the rest. The tests in `tests/` are short, readable examples of every behaviour (they use fake models that paint the frame_id into the image, which proves results are attached to the right frame).

Exercises: learn by changing things

1. Run `.\predict` and open `outputs\test\people\people_tracks.csv` in Excel. How many people were labelled unknown, and why might that be?
2. Move the counting line: `--count-line 0.5,0,0.5,1` (a vertical line). How do IN/OUT change?
3. Compare `--detect-every 1` and `--detect-every 3` on `--max-seconds 20`. Compare speed and handbags per frame in analytics_summary.json.
4. Make a worker fail on purpose: `--inject-error-every-b 10` (see `--help`). Find the red frames in the video and the ERROR rows in frames_log.csv.
5. Read `tests/test_reorder_association.py`, then write a test where results arrive in reverse order.
6. Run with `-v` and read the end of the log: which writer stage takes the most ms/frame?

9. Glossary

Term	Meaning
Frame / FPS	One picture of the video / frames per second (this video: 30)
Inference	Running a trained model on new data to get a prediction
YOLO	"You Only Look Once": a family of fast object detectors (here YOLO11 from Ultralytics)
Bounding box	Rectangle (x1, y1, x2, y2) around a detected object
Confidence / threshold	How sure the model is (0-1) / the minimum confidence that counts
COCO	Dataset of 80 everyday object classes the detector knows (person, handbag, car...)
ImageNet	Dataset of 1,000 classes used by the classifier; it has no "person" class
Gate	A cheap first model that decides whether the expensive model should run
Tracking / track ID	Linking detections over frames so the same object keeps one ID
ByteTrack	The tracking algorithm used (also keeps low-confidence boxes to survive occlusion)
Stride / HELD frame	Analysing only every Nth frame / a frame not analysed on purpose
Process / thread	Independent program with its own memory / a parallel path inside one process
GIL	Python lock that lets only one thread execute Python code at a time
Queue / backpressure	Hand-off buffer between stages / slowing the producer when buffers are full
Reorder buffer	Holds finished frames until all earlier frames are done, so output stays in order
OpenVINO	Intel's inference engine, used here for the person-attributes model
x264 / CRF	H.264 video encoder / its quality setting (23 = good quality, small file)
Latency vs throughput	Time for one frame to get through / frames finished per second

Further reading in this repository: `README.md` (all options and measurements), `dummy.md` (short setup guide), `docs/architecture.md` (design and failure handling in depth).